

FOUNDATIONS OF LARGE LANGUAGE MODELS (LLMs)

MODULE 2

Syllabus

- Introduction to Transformers, Attention Mechanism, Prompt Engineering: Basic and advanced techniques, Role of LLMs in building AI agents, Ethical considerations in LLM usage

Introduction to Transformers

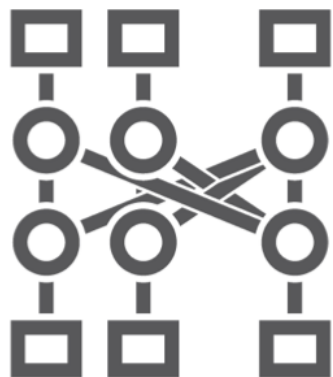
- The transformer model is a type of neural network architecture that excels at processing sequential data.
- It is a neural network that learns context and meaning by tracking relationships in sequential data like the words in a sentence.
- Transformers can train on entire texts all at once in parallel, rather than word-by-word in order. This makes training drastically faster.
- Transformers use attention mechanisms to understand each word in relation to all the other words in a sentence or document. This gives a richer understanding of context and meaning.

- The transformer layers analyze relationships between all the words simultaneously using self-attention.
- This allows them to model long-range connections in texts much better than older recurrent neural networks.
- After this context-aware analysis, the transformer layers then generate predictions or translations.
- The whole model is trained end-to-end on massive text datasets in a self-supervised way, learning by attempting to predict masked words.
- This breakthrough transformer architecture enables much higher performance on language tasks using less data and training time.
- Transformers power state-of-the-art results in machine translation, text generation, search, classification, and more.

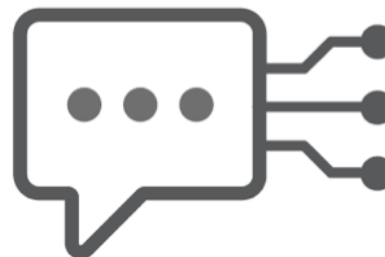
Key Features of Transformer Architecture:

- **Self-Attention Mechanism:** This is the core innovation of Transformers. It allows the model to weigh the importance of different words in a sequence when processing each word, regardless of their positions. This leads to a more efficient and effective handling of long-range dependencies.
- **Parallelization:** Unlike RNNs, which process data sequentially, Transformers can process all tokens (words or parts of a sentence) in parallel, significantly speeding up training and inference.
- **Encoder-Decoder Structure:** In models like BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pretrained Transformers), the Transformer typically operates in an encoder-decoder setup, where the encoder reads input data and the decoder generates output, such as text or sequences.
- **Long-range dependencies:** Directly connects any two positions, improving performance on tasks requiring context over long sequences.

Transformer Model



Applications



Natural Language
Processing (NLP)

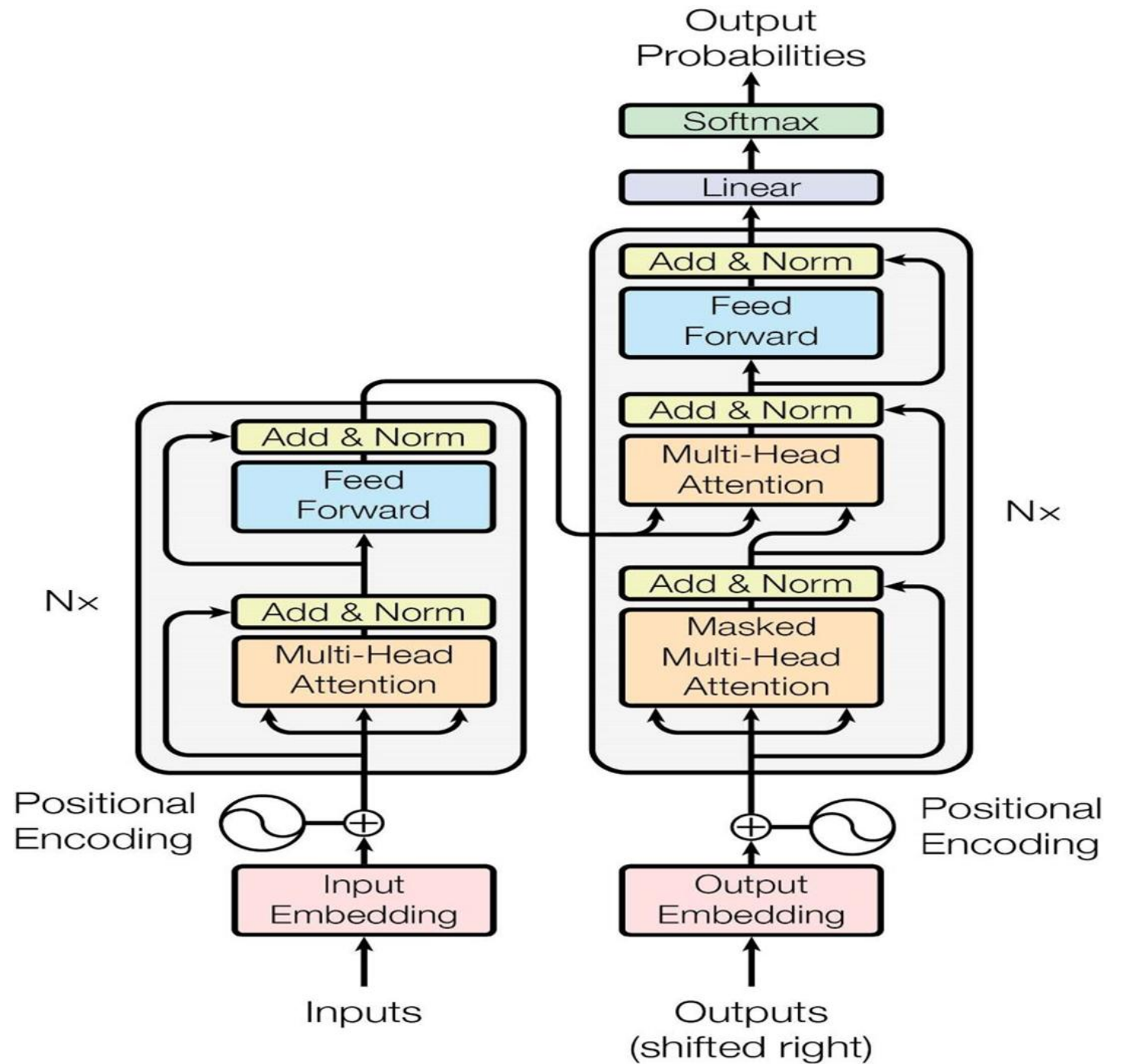


Computer
Vision



Time-Series
Forecasting

Architecture



- The Transformer architecture consists of an encoder and decoder, each composed of multiple identical layers

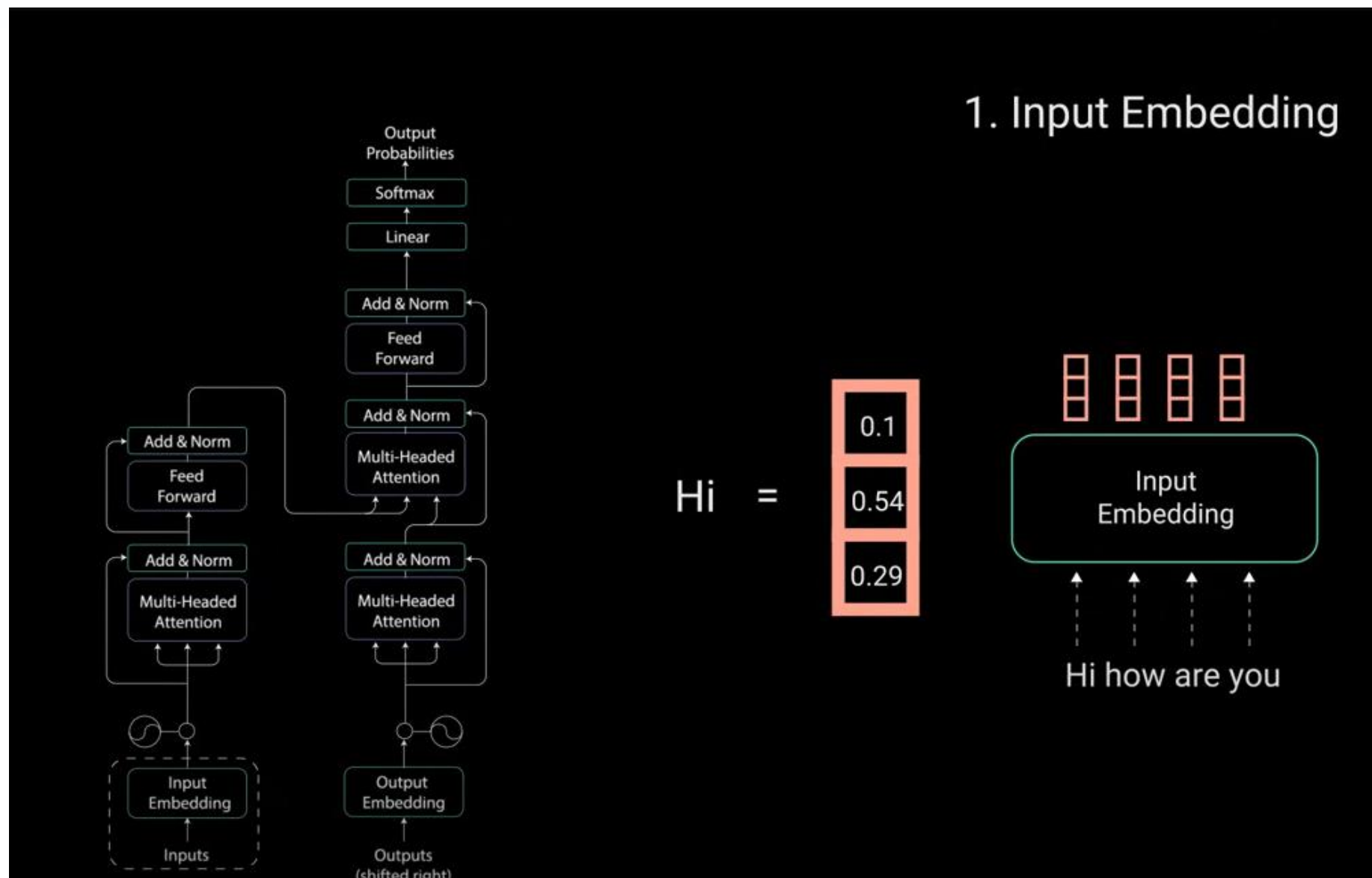
Component	Purpose	When to Use	Example
Input Embeddings	Convert tokens (words or subwords) into dense vectors of fixed size (e.g., 512 dimensions).	Essential for all Transformer models to represent input data numerically.	The word "cat" is mapped to a 512-dimensional vector capturing its semantic meaning.
Positional Encoding	Add information about token positions using sine and cosine functions, as Transformers lack inherent sequence order.	Critical for tasks where word order matters, like translation or summarization.	For "The cat is on the mat," positional encodings ensure "cat" is distinguished as the second word

Multi-Head Self-Attention	Allows each token to attend to all others, capturing diverse relationships (e.g., syntactic, semantic) via multiple attention heads (e.g., 8 heads).	Used in encoders for bidirectional context and decoders for autoregressive tasks.	In "The animal didn't cross the street because it was tired," one head may focus on "animal" and "it," another on "tired" and "didn't."
Masked Self-Attention	Prevents the decoder from attending to future tokens , ensuring autoregressive generation.	Essential for generative tasks like text generation or translation.	When generating "is" in "The cat is," the decoder cannot see "on the mat."
Encoder-Decoder Attention	Allows the decoder to attend to encoder outputs, aligning source and target sequences .	Used in sequence-to-sequence tasks like translation.	In translating to French, the decoder attends to "cat" in the encoder to generate "chat."
Feed-Forward Neural Networks	Apply position-wise non-linear transformations to enhance model capacity .	Used in every layer to process attention outputs	Transforms the attention output for "cat" to capture complex patterns.

Add and Norm (Residual Connections and Layer Normalization)	Stabilize training by adding input to output of sub-layers (residual connections) and normalizing layer outputs to maintain consistent scales.	Essential in every layer to prevent vanishing gradients and ensure stable training.	Adds the input of selfattention to its output, then normalizes to stabilize training for "cat."
Linear Layer	Maps the decoder's final hidden states to the vocabulary size for token prediction.	Used in the final decoder layer for generative tasks.	Maps a 512-dimensional hidden state to a 50,000-token vocabulary for predicting the next word.
Softmax	Converts linear layer outputs into probabilities over the vocabulary for token selection.	Used in the final decoder layer to produce a probability distribution for generative tasks.	Outputs probabilities over a 50,000-token vocabulary, selecting "is" as the next word in "The cat."

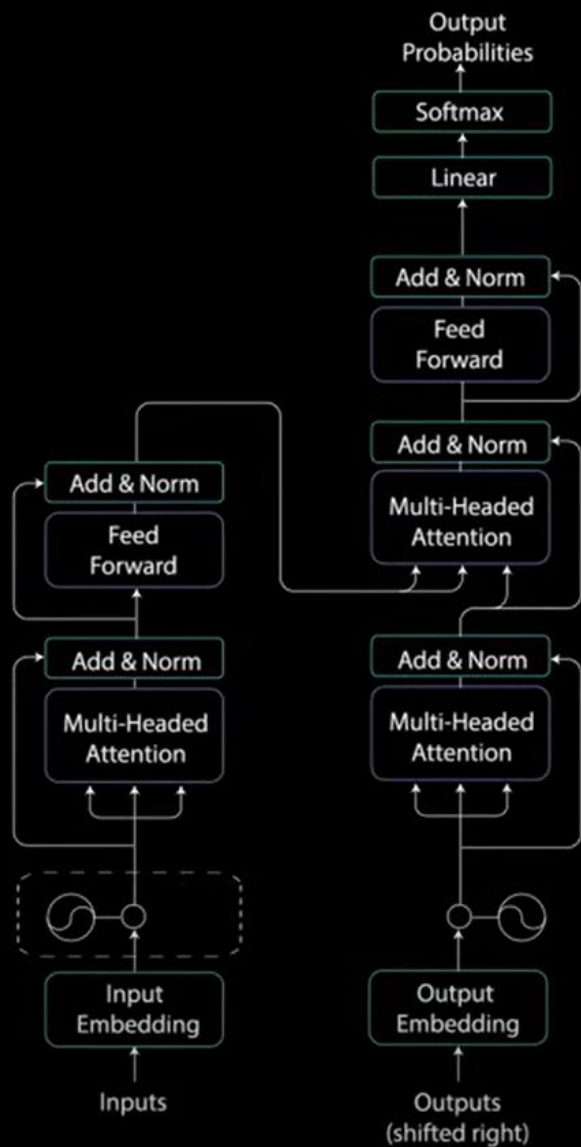


Transformer overview: attention, encoder–decoder structure.

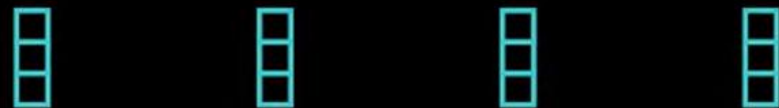




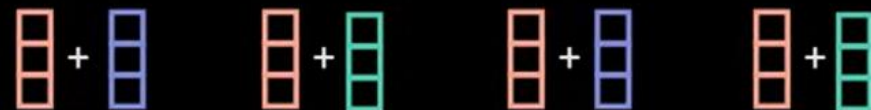
2. Positional Encoding



Positional Input
Embeddings



Positional
Encoding



Time Step

1

2

3

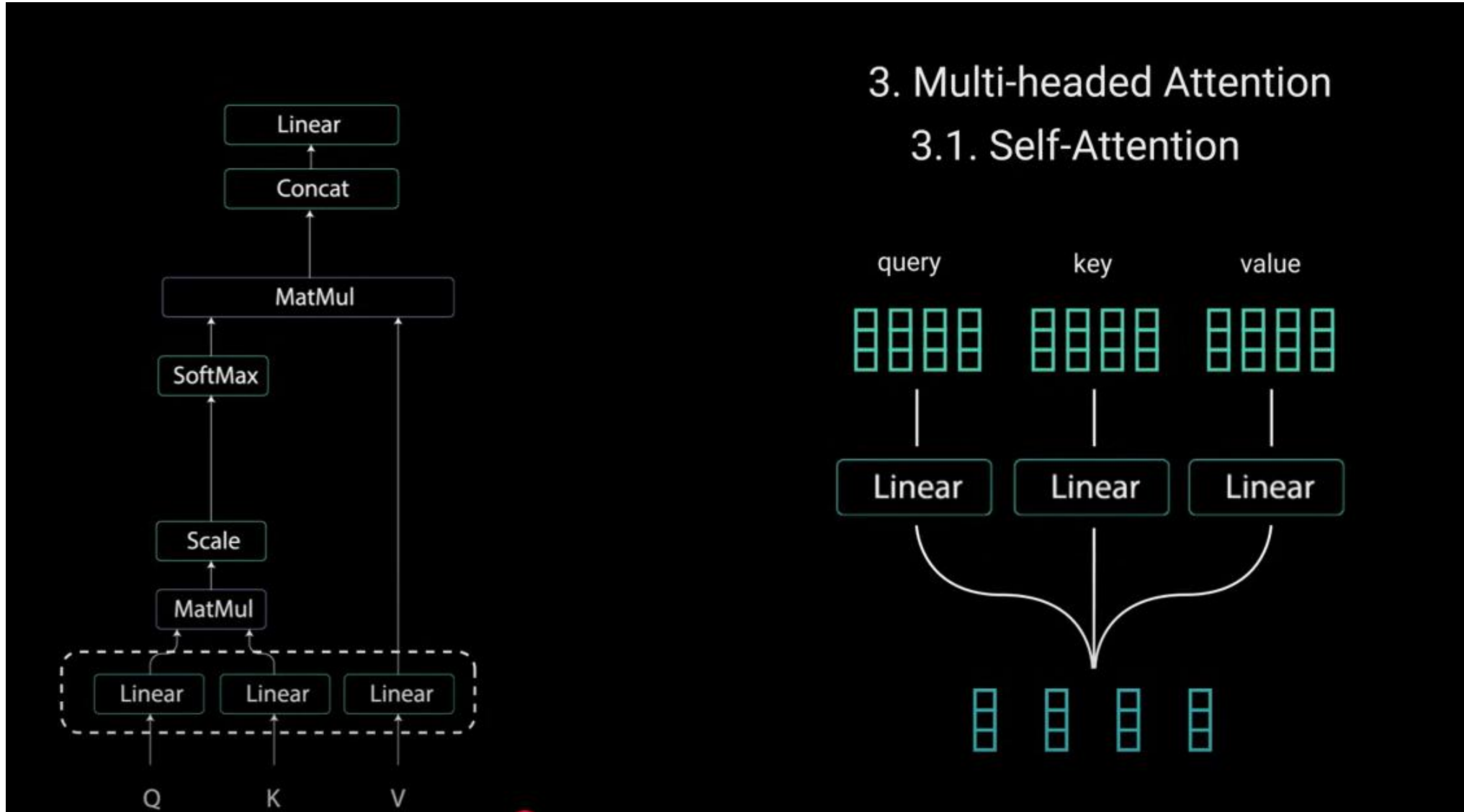
4

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$



Transformer overview: attention, encoder–decoder structure.



X **W^Q** **Q**

The diagram illustrates the matrix multiplication $X \times W^Q = Q$. Matrix X is a 2x4 grid of green squares. Matrix W^Q is a 4x3 grid of gray squares with a purple border. Matrix Q is a 2x3 grid of purple squares. The multiplication is indicated by a large 'x' between X and W^Q , and an equals sign between the result and Q .

X **W^K** **K**

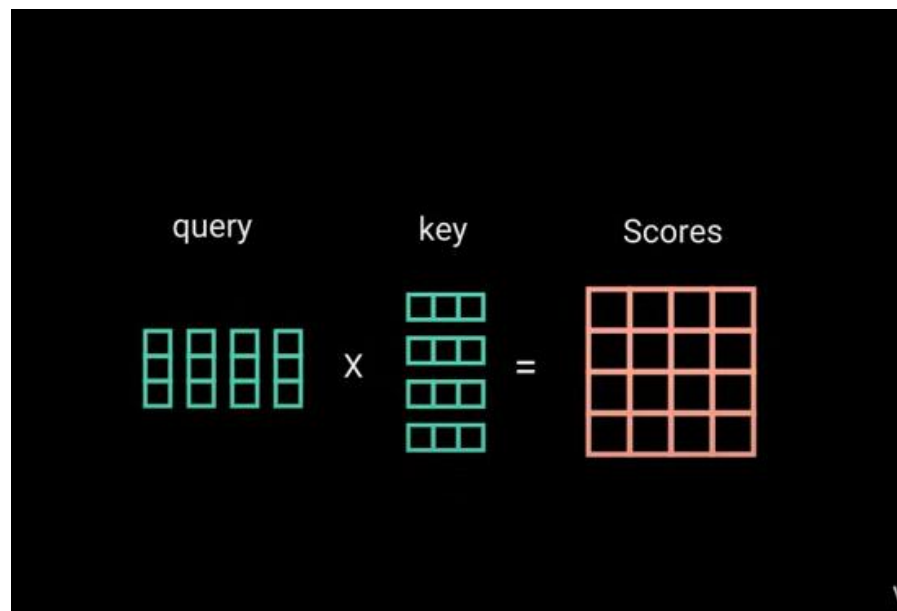
The diagram shows a green 2x4 matrix **X** multiplied by an orange 4x3 matrix **W^K**, resulting in an orange 2x3 matrix **K**. The matrices are represented by grids of colored squares: green for **X**, orange for **W^K** and **K**. The multiplication is indicated by a large 'x' and an equals sign.

X **W^v** **V**

The diagram illustrates the multiplication of two matrices. On the left is a green 2x4 matrix labeled **X**. In the middle is a blue 4x3 matrix labeled **W^v**. To the right of the green matrix is a multiplication symbol (×). To the right of the blue matrix is an equals sign (=). On the far right is a blue 2x3 matrix labeled **V**, which is the result of the multiplication.



Transformer overview: attention, encoder–decoder structure.



	Hi	how	are	you
Hi	98	27	10	12
how	27	89	31	67
are	10	31	91	54
you	12	67	54	92



Transformer overview: attention, encoder–decoder structure.

The diagram illustrates the scaling of attention scores. On the left, a 4x4 grid of orange-outlined squares represents the initial attention scores. Below this grid is a horizontal line, and underneath the line is the mathematical expression $\sqrt{d_k}$. To the right of the line and expression is an equals sign. To the right of the equals sign is another 4x4 grid, this one with teal-outlined squares, representing the scaled scores. The text "Scaled Scores" is positioned above this teal grid.

Scaled Scores

$$\frac{\text{Grid}}{\sqrt{d_k}} = \text{Scaled Scores Grid}$$



Transformer overview: attention, encoder–decoder structure.

3. Multi-headed Attention

3.1. Self-Attention

Softmax(

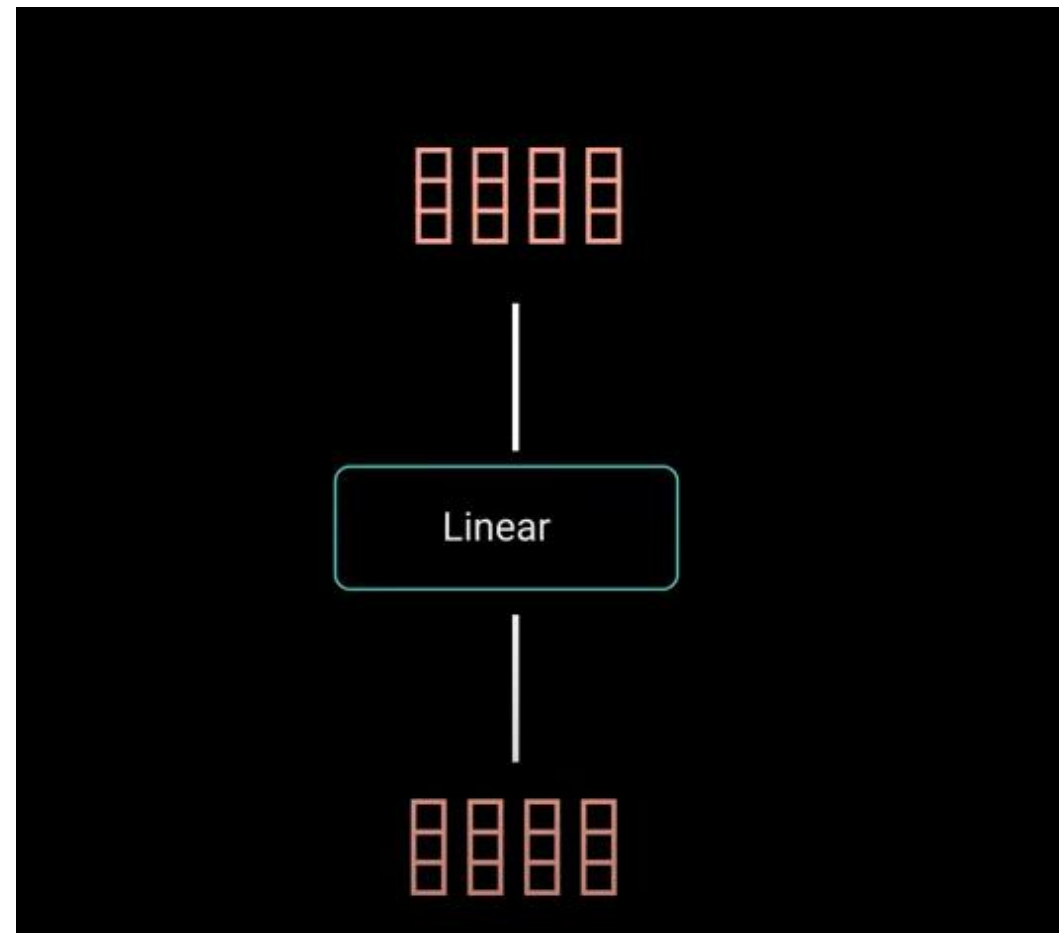
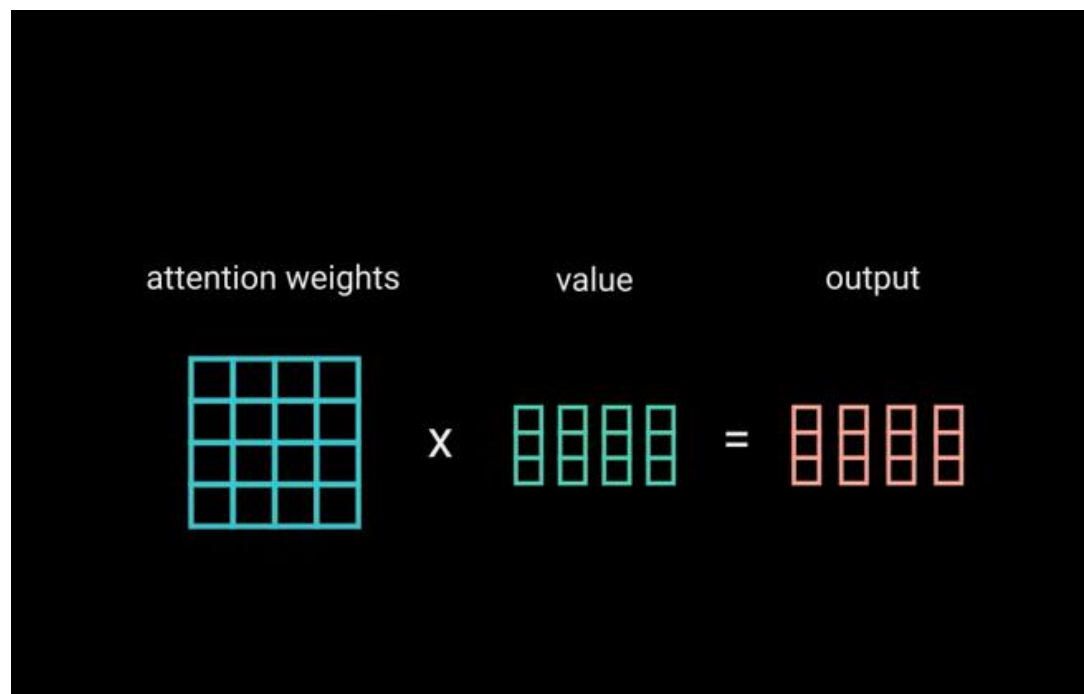
) =

	Hi	how	are	you
Hi	0.7	0.1	0.1	0.1
how	0.1	0.6	0.2	0.1
are	0.1	0.3	0.6	0.1
you	0.1	0.3	0.3	0.3

$$\text{softmax}(x)_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$



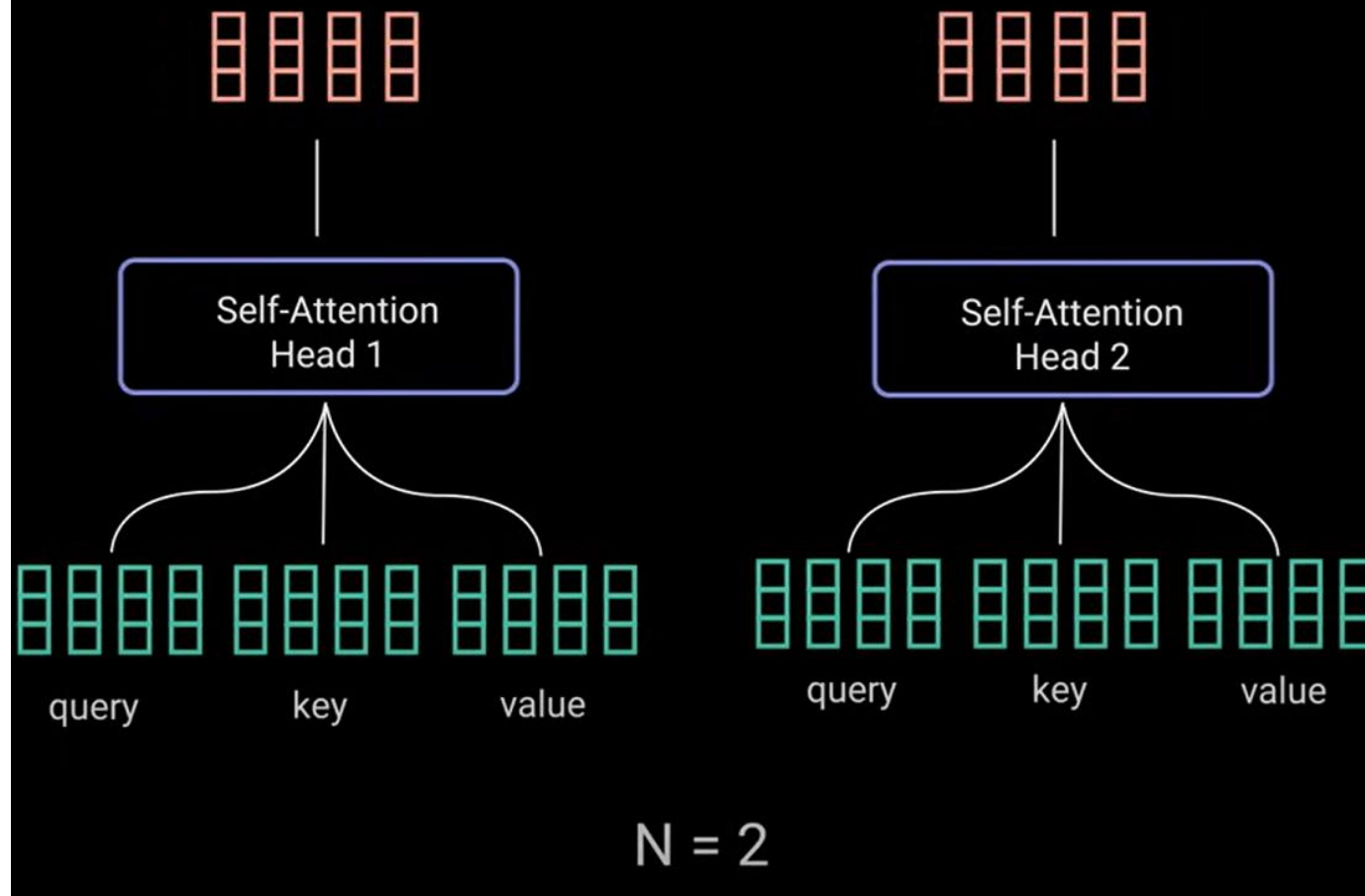
Transformer overview: attention, encoder–decoder structure.





Transformer overview: attention, encoder–decoder structure.

3. Multi-headed Attention



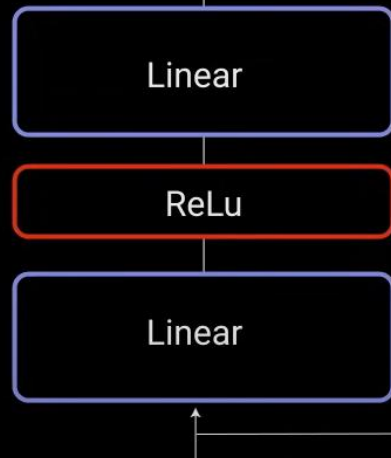
<https://www.youtube.com/watch?v=4Bdc55j80l8>



Transformer overview: attention, encoder–decoder structure.

4. Residual Connection, Layer Normalization & Pointwise Feed Forward

LayerNorm($\text{[vector]} + \text{[vector]}$)



LayerNorm($\text{[vector]} + \text{[vector]}$)

Output of the Encoder

The output of the final encoder layer is a set of vectors, each representing the input sequence with a rich contextual understanding. This output is then used as the input for the decoder in a Transformer model.

<https://www.youtube.com/watch?v=4Bdc55j80l8>

Masked Self-Attention Mechanism

scaled scores

0.5	0.2	0.1
0.1	0.6	0.2
0.1	0.2	0.3

+

Look-ahead mask

0	-inf	-inf
0	0	-inf
0	0	0

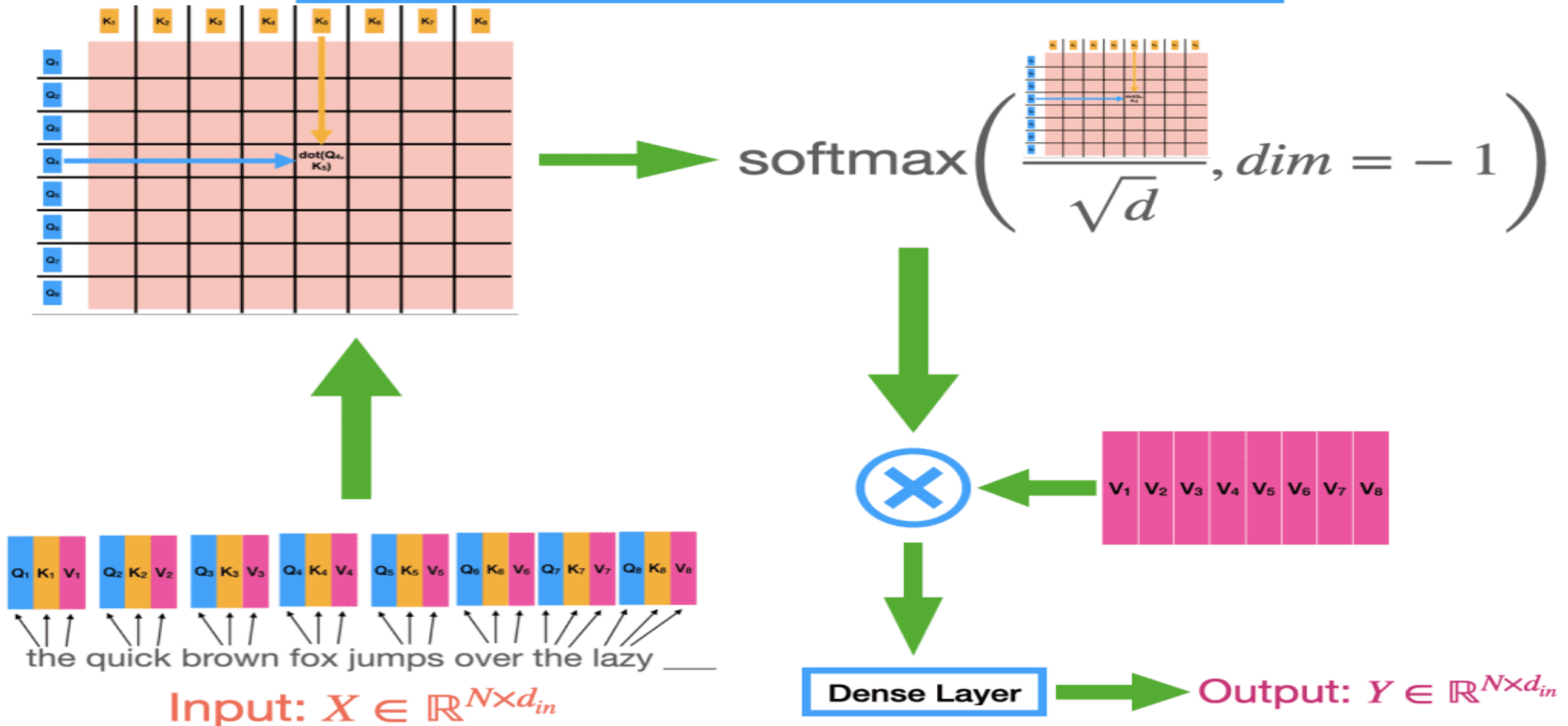
=

Masked Scores

0.5	-inf	-inf
0.1	0.6	-inf
0.1	0.2	0.3

Attention Mechanism

Neural Self Attention Mechanism



Prompt Engineering

- Prompt engineering that focuses on designing, refining, and optimizing prompts, i.e., inputs or instructions, to enable efficient and effective communication with AI models, especially large language models (LLMs).
- The goal is that AI systems generate outputs that are accurate, relevant, and aligned with user objectives.
- These prompts serve as a connection between the intentions of humans and the comprehension of machines. This allows AI models to generate outputs that are more precise and realistic.

What is a Prompt?

- A prompt is the instruction you give an AI model. The way you phrase it shapes the output.
- Bad Prompt:
 - “Tell me about AI.”
Output → A generic, textbook-like answer.
- Better Prompt:
 - “Explain AI to a 12-year-old using a pizza analogy in 3 bullet points.”
Output → Clear, structured, engaging.
- Prompt engineering is less about “tricking AI” and more about communicating clearly and intentionally.

Principles of Good Prompting

- Be Specific
- Assign Roles
- Step-by-Step Reasoning
- Apply Constraints
- Control Output Format

BAD PROMPT vs. GOOD PROMPT

BAD PROMPT



VS

BAD PROMPT



- A prompt contains any of the following elements:
- Instruction - a specific task or instruction you want the model to perform
- Context - external information or additional context that can steer the model to better responses
- Input Data - the input or question that we are interested to find a response for
- Output Indicator - the type or format of the output.

Understanding prompts

- A prompt is the input text or query provided to an AI model, such as a large language model to generate a response.
- It serves as the primary mechanism for guiding the model's behavior, defining the task and setting the context for the interaction.
- The design of a prompt significantly impacts the quality and relevance of the output, making it essential to choose the right type of prompt for specific tasks.
- To achieve the best results from AI models, it is essential to understand the various ways prompts can be structured to suit different tasks and objectives.
- There are three primary ways to structure the prompt: **direct instructions, open-ended instructions and task-specific instructions.**

Direct Instructions

Direct instructions are **clear and specific commands** that tell the AI exactly what to do.

These prompts are ideal for straightforward tasks where the user has a clear expectation of the output.

Direct prompts rely on the model's ability to parse explicit instructions and generate responses that align closely with the command.

The more detailed the instruction, the more likely the output will meet expectations.

Example:

Write a poem about nature.

In this case, the AI knows the exact format *[a poem]* and topic *[nature]* to generate the text.

Open-ended instructions

Open-ended instructions are less restrictive and encourage the AI to **explore broader ideas or provide creative and interpretive responses**.

These prompts are useful for brainstorming, storytelling or exploratory discussions where the user values variety and originality in the output.

Open-ended prompts tap into the model's generative capabilities without imposing constraints. The model relies on its training data to infer the best approach to the prompt, which can produce diverse or unexpected results.

Example:

Tell me about the universe.

Here, the AI has the freedom to decide what aspects of the universe to discuss, such as its *origin, structure or scientific theories*.

Task-specific instructions

- **Task-specific instructions** are designed for precise, **goal-oriented tasks**, such as translations, summarization or calculations.
- These prompts are often crafted with clarity and can include additional context or examples to help ensure accurate responses.
- **Task-specific prompts** leverage the model's understanding of specialized tasks. They can incorporate advanced prompting techniques like few-shot prompting (providing examples) or zero-shot prompting (providing no examples but relying on the model's pretrained knowledge).
- Example:
- Translate this text into French: 'Hello.'
- The model understands both the language translation task and the specific input text, enabling it to produce the desired output: "Bonjour."

Key techniques in prompt engineering

- To maximize the effectiveness of AI models, prompt engineering employs a variety of techniques tailored to different tasks and objectives.

Zero-shot prompting

- Zero-shot prompting involves asking the model to perform a task without providing any prior examples or guidance. It relies entirely on the AI's pretrained knowledge to interpret and respond to the prompt.
- Example prompt:

Explain the concept of climate change, its causes, and its effects in simple terms.

The model is given no prior examples or additional context and must rely solely on its pretrained knowledge to generate the output.

Few-shot prompting

- Few-shot prompting includes a small number of examples within the prompt to demonstrate the task to the model. This approach helps the model better understand the context and expected output.
- Example prompt:
- Here are some examples of how to explain complex topics:
 - - Topic: Photosynthesis
 - - Explanation: Photosynthesis is the process by which plants convert sunlight, water, and carbon dioxide into energy and oxygen.
 - - Topic: Gravity
 - - Explanation: Gravity is the force that pulls objects toward each other, like how the Earth pulls us to its surface.
 - Now explain: Climate Change.

By providing a few examples of how to explain other topics, the model is guided on the tone and level of simplicity expected for the climate change explanation.

Chain of thought (CoT) prompting

[CoT prompting](#) encourages the model to reason through a problem step by step, breaking it into smaller components to arrive at a logical conclusion.

Example prompt:

Step 1: Define what climate change is.

Step 2: Explain the causes of climate change.

Step 3: Describe its effects on the planet.

Now, follow these steps to explain climate change.

The model is encouraged to think step by step, breaking down the explanation into smaller, logical parts for clarity.

Meta prompting

- Meta prompting involves asking the model to generate or refine its own prompts to better perform the task. This technique can improve output quality by leveraging the model's ability to self-direct.
- Example prompt:
- Create a prompt that will help you explain climate change, its causes, and its effects in simple terms
- The model generates its own prompt before attempting to explain the topic, potentially improving the relevance and quality of the output.

Self-consistency

- Self-consistency uses multiple independent generations from the model to identify the most coherent or accurate response. It's particularly useful for tasks requiring reasoning or interpretation
- Example prompt:
 - Provide three different explanations of climate change, its causes, and its effects. Then identify the most coherent and clear explanation
- The model produces multiple independent responses and selects the most consistent or coherent one as the final output.

Generate knowledge prompting

- This technique involves asking the model to generate background knowledge before addressing the main task, enhancing its ability to produce informed and accurate responses.
- Example prompt:

Before explaining climate change, first list the key scientific principles related to it. Once done, use these principles to explain the concept, its causes, and its effects.

The model generates background knowledge first (for example, greenhouse gases, global warming) to provide a more informed explanation.

Prompt chaining

- Prompt chaining involves linking multiple prompts together, where the output of one prompt serves as the input for the next. This technique is ideal for multistep processes.
- Example prompt:

What is climate change? Provide a brief definition.

Next prompt based on the previous response:

What are the primary causes of climate change?

Next prompt based on the previous response:

What are the effects of climate change on the environment and human life?

The task is divided into a chain of smaller prompts, with the output of each step feeding into the next for a more structured explanation.

Tree of thoughts prompting

- [Tree of thoughts prompting](#) encourages the model to explore multiple branches of reasoning or ideas before arriving at a final output.
- Example prompt:
- List three possible ways to explain climate change to a general audience. For each method, describe its advantages and disadvantages. Then choose the best explanation and elaborate on it
- The model explores multiple approaches to the explanation and selects the most effective one, providing a well-rounded output.

Retrieval augmented generation (RAG)

- Retrieval augmented generation (RAG) combines external information retrieval with generative AI to produce responses based on up-to-date or domain-specific knowledge.
- Example prompt:
- Using the global temperature datasets from NASA GISS (GISTEMP) dataset on climate science, explain climate change, its causes, and its effects in simple terms.
- The model combines its generative abilities with external knowledge to produce an informed explanation.

Automatic reasoning and tool-use

- This technique integrates reasoning capabilities with external tools or application programming interfaces (APIs), allowing the model to use resources like calculators or search engines.
- Example prompt:
- Use the provided climate data to calculate the global temperature rise over the last century, and then explain how this relates to climate change, its causes, and its effects.
- The model integrates reasoning with external tools (for example, calculators or APIs) to analyze data and provide a data-driven explanation.

Automatic prompt engineer

- This method involves using the AI itself to generate and optimize prompts for specific tasks, automating the process of crafting effective instructions.
- Example prompt:
- Generate a prompt that will help explain climate change, its causes, and effects. Then use the generated prompt to provide the explanation.
- The model automates the creation of an optimized prompt to improve the quality of its response.

Active-prompt

- Active-prompting dynamically adjusts the prompt based on intermediate outputs from the model, refining the input for better results.
- Initial prompt
- Explain climate change, its causes, and its effects in simple terms.
- Follow-up prompt
- Add more detail about the causes of climate change, focusing on human activities.

The prompt dynamically evolves based on the intermediate output, refining the response over iterations

Directional stimulus prompting

- Directional stimulus prompting (DSP) uses directional cues to nudge the model toward a specific type of response or perspective.
- Example prompt:
- Explain the concept of climate change from an environmentalist's perspective, focusing on the need for immediate action.
- The model is nudged toward a specific perspective or tone, influencing the framing of its explanation

Program-aided language models (PALM)

- PALM integrates programming capabilities to augment the model's reasoning and computational skills.
- Example prompt:
- Write Python code to visualize the increase in global temperatures over time. Then explain how this data relates to climate change, its causes, and its effects.
- The model combines programming with language generation to provide both a visualization and an explanation.

ReAct

- [ReAct](#) combines reasoning and acting prompts, encouraging the model to think critically and act based on its reasoning.
- Example prompt:
- Analyze the following climate data and identify key trends. Based on your analysis, explain the concept of climate change, its causes, and its effects.
- This example illustrates how the model can combine analytical reasoning with actionable insights.

Reflexion

- Reflexion allows the model to evaluate its previous outputs and refine them for improved accuracy or coherence
- Example prompt:
 - Here is my first attempt at explaining climate change:
 - [Insert initial output].
 - Review this explanation and improve it for clarity and accuracy.

The model reflects on its previous output and iteratively improves it

Multimodal chain of thought (multimodal CoT)

- This technique integrates chain of thought reasoning across multiple modalities, such as text, images or audio.
- Example prompt:
- Analyze this infographic on global warming trends, then explain climate change, its causes, and its effects step by step
- The model integrates reasoning across multiple modalities (text and images) to deliver a comprehensive explanation.

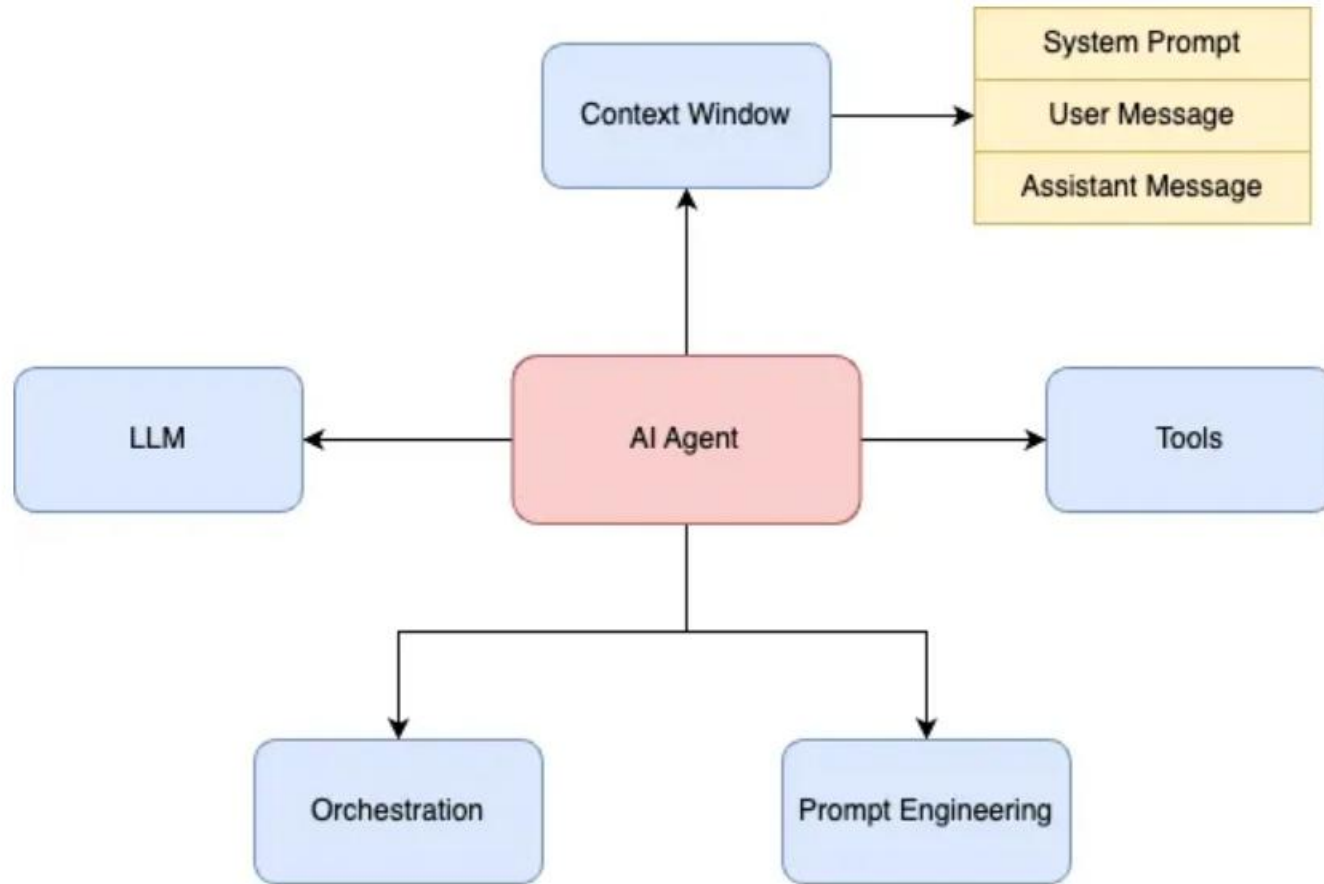
Graph prompting

- Graph prompting leverages graph-based structures to organize and reason through complex relationships between concepts or data points.
- Example prompt:
- Using the provided graph of CO₂ emissions over time, explain how it relates to climate change, its causes, and its effects.
- The model uses graph-based reasoning to connect data points and generate an insightful explanation.

What is an AI Agent?

- An AI Agent is a software powerhouse powered by artificial intelligence, designed to autonomously perform a wide array of tasks using LLMs (Large Language Models).
- AI agents can be your personal coders, tireless researchers, or even creative problem solvers. Here's what makes them exceptional:
 - Natural Language Understanding: Effortlessly process and act on conversational commands.
 - Task Automation: Eliminate the monotony by handling repetitive tasks with precision.
 - Seamless Integration: Connect with APIs, databases, or external systems to amplify capabilities.
 - Continuous Learning: Adapt to feedback and evolve to deliver better results over time.
 - Collaborative Intelligence: Work with multiple tools or even other agents to accomplish complex workflows.

Before building agents, let's clarify some key concepts:



- Example orchestration flow:

User input → LLM processing → Context retrieval → Tool selection →
Execution → Response generation

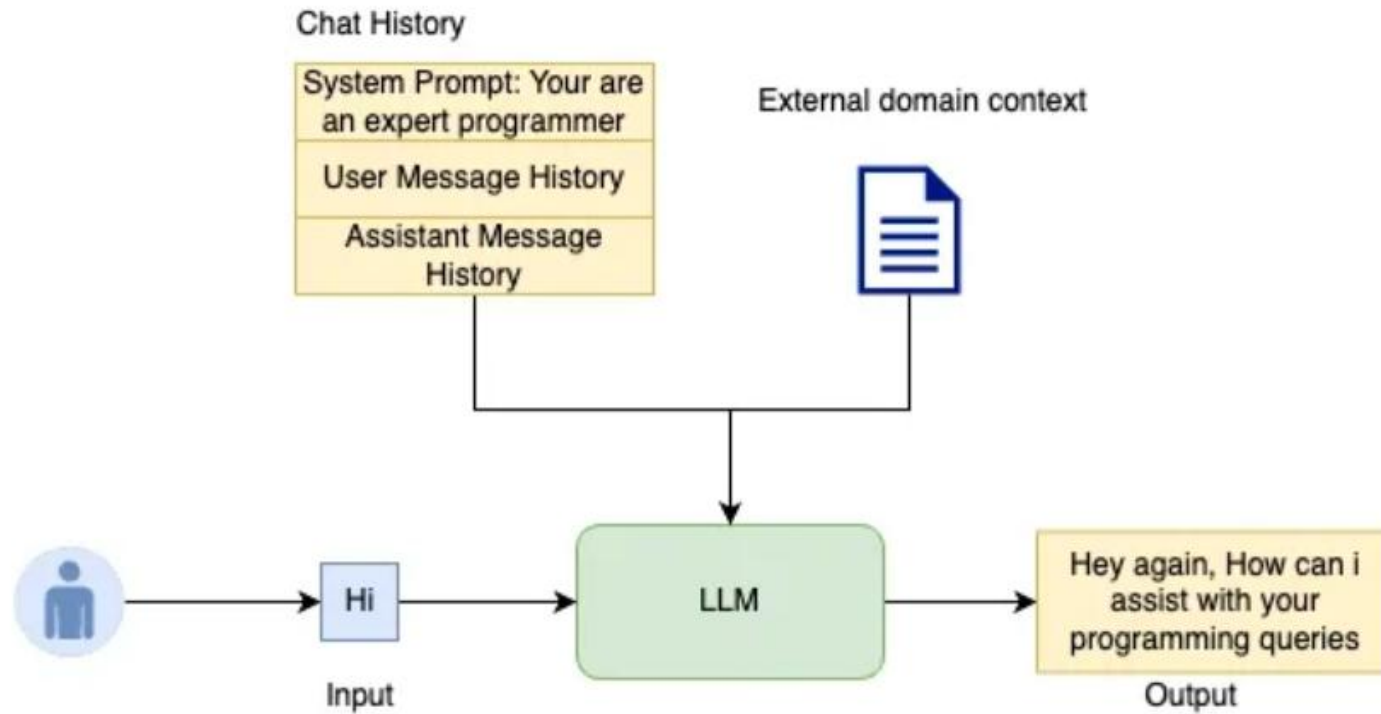
Understanding Integration Levels

- When building AI agents, it's crucial to understand the different ways we can interact with LLMs. Each level of integration adds new capabilities and complexity, allowing us to create increasingly sophisticated agents. The progression follows a natural path:
 1. Basic Integration: Simple request-response pattern
 2. Contextual Integration: Adding memory and personality
 3. Tool Integration: Extending capabilities beyond text
 4. Self-Reflection: Adding reasoning and improvement

Basic LLM Integration

- The simplest form of interaction is a direct request-response pattern. This is the foundation of all LLM interactions. At this level, we focus on:
 - Message Structure: Understanding how to format requests
 - Response Handling: Processing and displaying responses
 - Error Management: Handling potential failures
 - Performance Tracking: Monitoring token usage and completion reasons

Adding Context

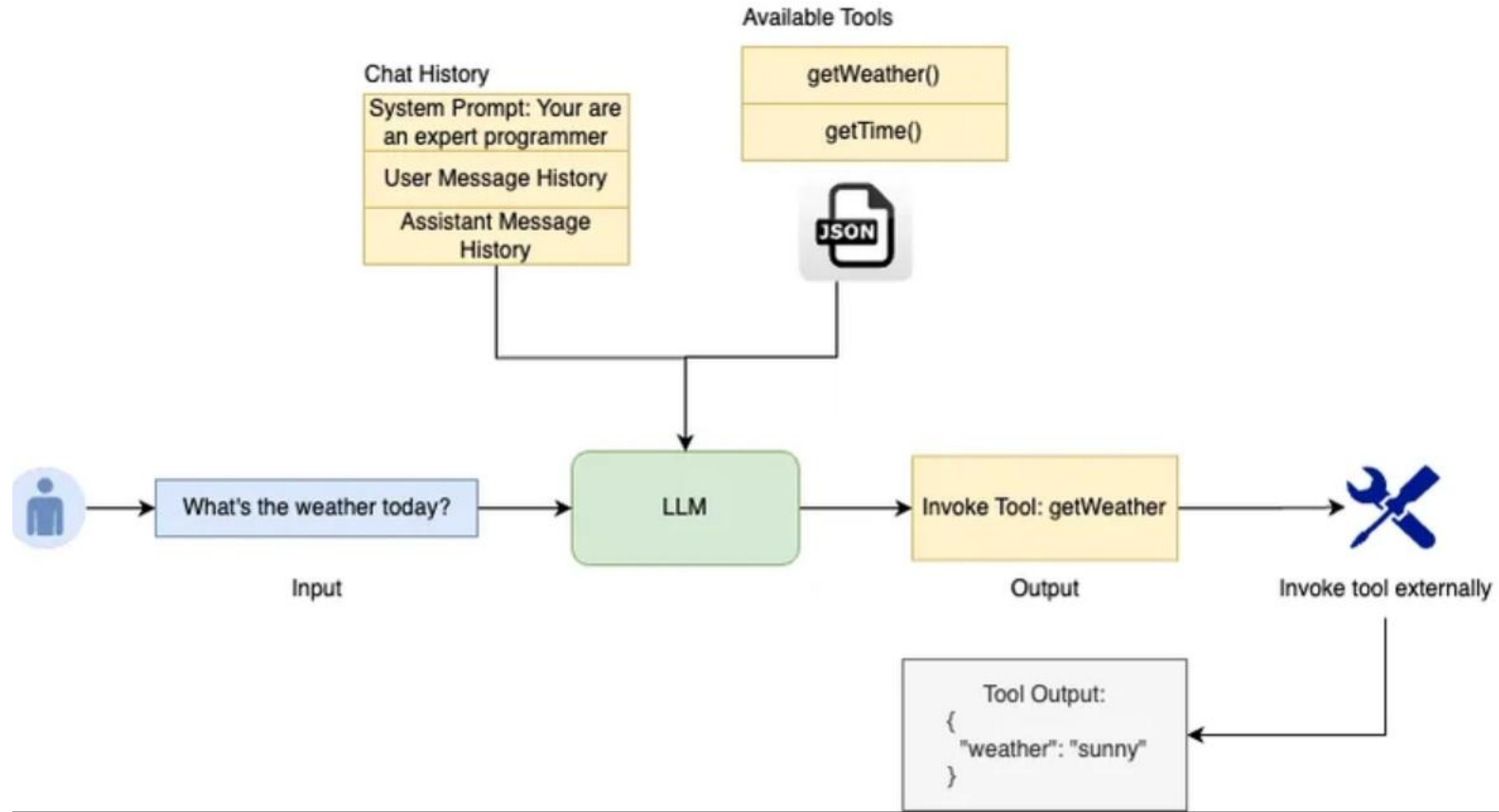


Context enriches our interactions by providing additional information or setting specific behaviors for the LLM.

This level introduces:

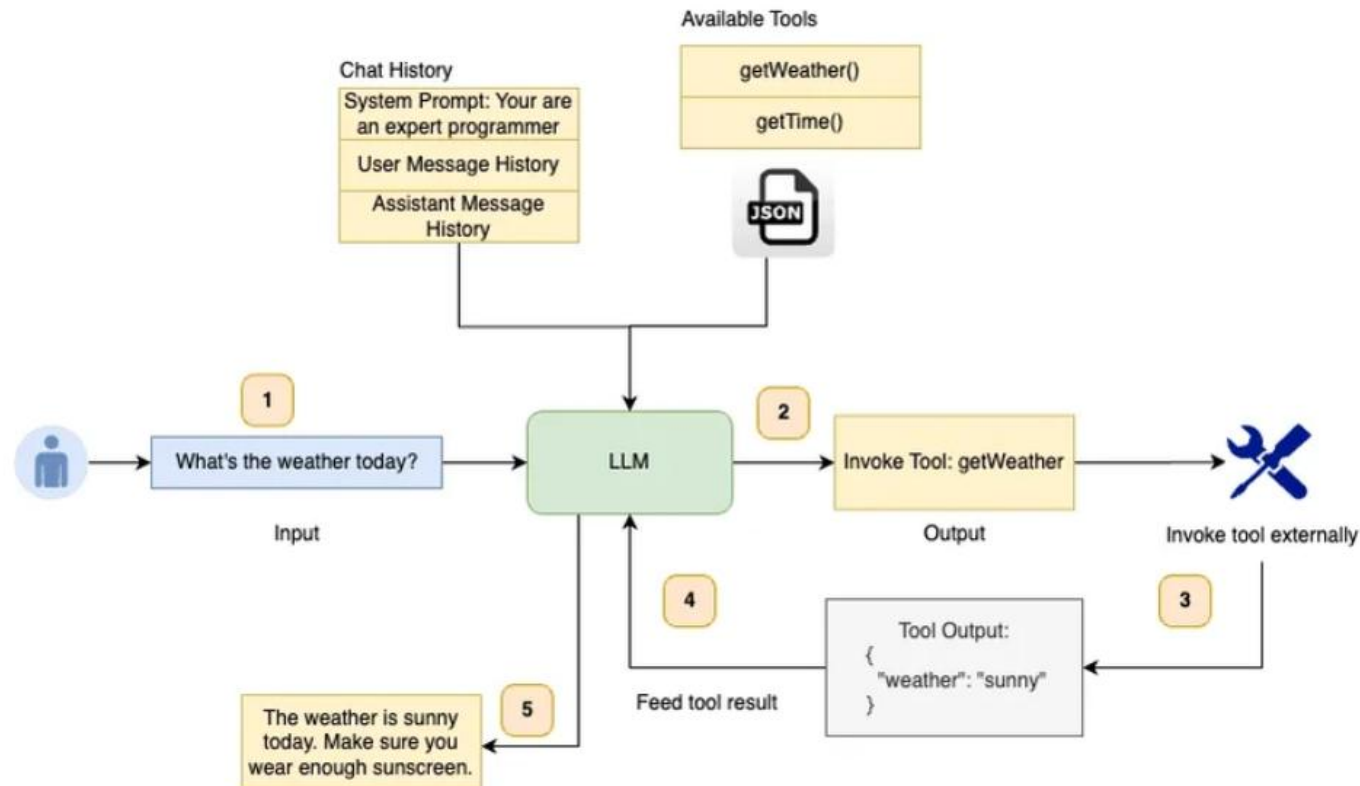
- System Prompts: Setting behavior and personality
- Conversation History: Maintaining context across interactions
- Role-Based Interactions: Different types of messages
- Enhanced Response Control: More precise outputs

Tool Integration



- Tools extend an agent's capabilities beyond text generation, allowing it to perform specific tasks or access external resources. This level introduces:
 - Tool Definition: Structured way to define capabilities
 - Tool Selection: LLM decides when to use tools
 - Result Integration: Incorporating tool outputs
 - Error Handling: Managing tool execution failures

Self-Reflection and Improvement



- The most sophisticated level adds self-reflection through continuous interaction and tool usage. This creates a more dynamic and capable agent that can:
- Maintain Context: Keep track of conversation state
- Chain Actions: Combine multiple steps
- Self-Correct: Identify and fix issues
- Learn from Feedback: Improve responses based on results

The role of LLMs in AI agents?

- LLMs equip AI agents to understand language, analyze data, and act precisely, driving more intelligent decisions and efficient operations for your business

1. Enhance natural language understanding (NLU) for seamless interaction

- Context awareness. Maintain continuity across conversations.
- Personalization. Adapt tone to each customer.
- Sentiment recognition. Identify emotions in real time.
- Multilingual support. Connect with diverse audiences.

These improvements set the stage for stronger reasoning and workflow problem-solving.

2. Enable advanced reasoning to solve complex problems

- Pattern recognition. Identify trends across datasets.
- Scenario analysis. Weigh outcomes before acting.
- Adaptive responses. Adjust to new inputs.
- Resource optimization. Direct effort effectively.

These capabilities lay the groundwork for stronger workflow planning and precise execution

3. Optimize workflow planning and task execution

- Task orchestration. Organize linked steps and handoffs.
- Priority scheduling. Rank tasks by impact and deadlines.
- Fault handling. Detect errors and trigger safe rollbacks.
- Performance tracking. Monitor throughput and cycle times.

This focus on planning prepares your team for precise task execution and smoother handoffs across your systems.

4. Integrate with external tools and system connections

- Customer relationship management (CRM) access. Retrieve records instantly.
- Calendar sync. Book meetings automatically.
- Payment gateways. Process transactions securely.
- Analytics dashboards. Deliver live insights.

This synergy allows AI agents to operate across platforms, expanding impact across workflows.

5. Manage memory and knowledge for long-term learning

- Context recall. Retain details across interactions.
- Knowledge updates. Absorb new data for accuracy.
- Experience-based refinement. Improve outputs through patterns.
- Policy alignment. Apply rules consistently in operations.
- By combining stored knowledge with ongoing input, AI agents build dependable intelligence that grows with your workflows.

6. Fine-tune LLMs for domain-specific agent tasks

- Industry alignment. Match vocabulary and workflows to your sector.
- Regulatory compliance. Follow rules relevant to your business.
- Customer focus. Respond using terms familiar to your audience.
- Process fit. Adapt to unique operational routines.
- By training models with targeted datasets, agents deliver responses aligned with your goals and workflows.

7. Strengthen security and governance in AI agent operations

- Access controls. Limit permissions to protect sensitive records.
- Data masking. Shield personal details during processing.
- Audit trails. Track agent activity for accountability.
- Policy enforcement. Align actions with organizational standards.
- By embedding these protections, AI agents handle automation without compromising trust

Ethical considerations in LLM usage

- Privacy and Data Usage
- Bias and Fairness
- Misinformation and Manipulation
- Economic and Social Impact
- Accountability and Transparency
- Environmental Concerns

Potential Solutions and Best Practices

- Ethical AI Frameworks: Develop and adhere to comprehensive ethical guidelines for AI development and deployment.
- Diverse Development Teams: Ensure AI teams are diverse to help identify and mitigate potential biases.
- Ongoing Monitoring and Auditing: Implement systems for continuous evaluation of LLM outputs and impacts.
- Transparency Measures: Provide clear information about an LLM's capabilities, limitations, and potential risks.
- Collaborative Governance: Foster cooperation between tech companies, policymakers, and ethicists to develop effective AI governance structures.
- Education and Awareness: Promote AI literacy to help users understand and critically evaluate LLM outputs